Oct 30 2026

VanBenthem1986_Book_EssaysInLogicalSemantics.pdf : Natural Logic, Categorical Grammar and Formal logic: (Finding intersection to Natural Logic and FOL for NLI and)

1. Natural Language Inference (with a subset of Natural Logic

Natural Language Inference (with a subset of Natural Logic) and First Order Logic:

The procedure to verify whether a hypothesis is entailed from premise or not and whether it is a contradiction using FOL is possible.

Natural Logic	First Order Logic (basic configuration)	FOL Resolution Configuration for checking if it is entailment (i.e. valid = derives empty clause)	Not empty clause: FOL Resolution for checking if it is Contradiction	Neither resolved to empty clause: Then it is neutral
Premise P	KB = P	KB	KB	Run both tests and Neither yields empty clause, then Neutral.
Hypothesis H	Query = H	¬H	H	
These are the equivalence procedures for “strict entailment” and “strict contradiction”. We often have systems that only look for the “strict” or “absolute” truths.				

Incoherent Universals

What do Incoherent Universals really do?

- P = KB = Bat is a mammal. Bat is used in baseball. Bats have wings. Bats have a barrel. Wings are not barrels. Barrels are not wings. Bats can fly. There is a Brown bat.
- H = Query = Brown Bat is a mammal.
- LLMs ground incoherent universals : i.e. Brown bat to a mammal (based on ranked answer retrieval, which most recent research points to). But it is not a requirement for FOL. FOL does not require LLM to ground its common sense.
- Resolution procedure requires to convert to Conjunctive Normal Form or Clause Normal Form (CNF) - which orders the clauses. The resolution procedure can pick up any clause and may end up with empty clause whether it takes direction of "
  $\forall x_{Animal}(Bat(x_{Animal}) \rightarrow HasWings(x_{Animal}))$

  $\forall y_{Sports}(Bat(y_{Sports}) \rightarrow HasBarrel(y_{Sports}))$

- By Symbolic Logic proof method: Domain of discourse D:
  - no n empty set of abstract mathematical objects d1, d2 etc
  - If c is a constant, Interpretation I(c) belongs to Domain D.
  - If P is an unary predicate (note this P is different from premise P), I(P) is a subset of Domain D.
  - **Construct Mapping function I :**
    - Constants: Brown_Bat
    - Predicates: Bat(x), HasWings(x), HasBarrel(x)
    - Domain = {d1, d2}
    - I(Brown_bat) = d1
    - Predicates:
      - $I(Bat) = \{d_1, d_2\}$ *(Elements d_1 and d_2 are "bats").*
      - $I(HasWings) = \{d_1\}$ *(Only element d_1 has wings).*
      - $I(HasBarrel) = \{d_2\}$ *(Element d_2 has barrels).*
    - Evaluating truth under this Interpretation
      - Rule 1: "Wings are not barrels" (
        $\forall x(HasWings(x) \rightarrow \neg HasBarrel(x)))$
        - **Mathematical Translation:**
          $I(HasWings) \cap I(HasBarrel) = \Phi$

        - **Evaluation:** $\{d_1\} \cap \{d_2\} = \Phi$ **and the rule evaluates to TRUE.**
    - Rule 2: "Bats have wings" ($\forall x(Bat(x) \rightarrow HasWings(x))$)
        - **Mathematical Translation:** The set of bats must be a subset of the set of things with wings ($I(Bat) \subseteq I(HasWings)$).
        - **Evaluation:** Is $\{d_1, d_2\} \subseteq \{d_1\}$? **No.** Element d_2 is a bat, but it does not have wings. **This rule evaluates to FALSE.**
    - Rule 3: "Bats have a barrel" ($\forall x(Bat(x) \rightarrow HasBarrel(x))$)
        - **Mathematical Translation:** The set of bats must be a subset of the set of things with barrels ($I(Bat) \subseteq I(HasBarrel)$).
        - **Evaluation**: Is $\{d_1, d_2\} \subseteq \{d_2\}$? No. Element d_1 is a bat, but it does not have a barrel. This rule evaluates to FALSE.
    - Rule 4: "Barrels are not wings" ($\forall x(HasBarrel(x) \rightarrow \neg HasWings(x))$)
        - **Mathematical Translation:** $I(HasBarrel) \cap I(HasWings) = \Phi$
        - **Evaluation:** $\{d_1\} \cap \{d_2\} = \Phi$ **and the rule evaluates to TRUE. (Intersection is associative, but still for FOL's construction sake, we repeated).**
    - The catch with Structural Reality of Incoherence:
        - If rule 2 needs to be true, d2 also must have wings, but if it is true, rule 2 gets violated and becomes false since wings are not barrels.
        - If rule 3 is true, d1 has barrel, which makes rule 1 becomes false (wings are not barrels)
    - Forcing any interpretation, makes rules 2 and 3 true simultaneously, breaks rule 1 and there is no possible mathematical model for this.
- The clause ordering from the Clausal Normal Forms (or Conjunctive Normal forms) create a clause search space. It is valid that the resolution procedure does not have to traverse all of the CNFs in search space to derive an empty clause. This means that without traversing the actual query clause, resolution can derive the empty

clause - which is the problem with resolution. This is the reason NLI engines since probably right before BERT, have been using Set of Support i.e. only select pairs of clauses for resolution that contain atleast one of the clauses descends from the Hypothesis (or The Query).

- Thus the world created by this KB is mathematically impossible i.e unsatisfiable.
- Even though resolution is an oversaturated search as new clauses keep getting added to the pool of CNFs, so far an empty clause is not yet derived.
- This is an **unrestricted resolution.** This leads to Linear resolution.

- **Linear Resolution:**
  - In this linear resolution, a resolution step has to be linear i.e. it cannot break and create new branch without following the update:
    - Last derived clause (center clause), An original or previous clause (side clause) derives new center clause.
    - This forms a chain.
  - Let us consider the CNFs for this KB, query:
  - C1: $\neg Bat(x) \vee HasWings(x)$
  - C2: $\neg Bat(y) \vee HasBarrel(y)$
  - C3: $\neg HasWings(z) \vee \neg HasBarrel(z)$
  - C4: $Bat(Brown_Bat)$
  - C5: $\neg CanFly(Brown_Bat)$ (The Negated Query / Hypothesis Test)
  - In an unrestricted resolution, the engine can pick any clause. But with set of support, forces engine to always look for clauses in the query, which can make it struck and never resolve or internal non-query clauses first and resolves without ever touching clause in query i.e. whether Brown Bat is a mammal requires to satisfy that it can fly.
- **Steps for unrestricted resolution:**
  1. Step 1: Resolve C1 and C4. The engine matches $\neg Bat(x)$ with $Bat(Brown_Bat)$. Substitution: \(\{x \mapsto Brown_Bat\}\) Derived Clause C6: $HasWings(Brown_Bat)$
  2. Step 2: Resolve C2 and C4. The engine matches $\neg Bat(y)$ with $Bat(Brown_Bat)$. Substitution: \(\{y \mapsto Brown_Bat\}\) Derived Clause C7: $HasBarrel(Brown_Bat)$
  3. The clause pool now contains our derived facts about our abstract domain element, Brown_Bat: C3:

$\neg HasWings(z) \vee \neg HasBarrel(z)$C6: $HasWings(Brown_Bat)$C7: $HasBarrel(Brown_Bat)$

4. Step 3: Triggering the Contradiction The engine resolves C3 and C6. (substitutes \(\{z \mapsto Brown_Bat\}\)), yielding C8: $\neg HasBarrel(Brown_Bat)$. The engine immediately resolves C8 with C7 ($\neg HasBarrel$ vs $HasBarrel$). Result: Empty Clause (\[\Box \]).
5. But derives hypothesis true but without actually ever using C5 clause.

- **Steps for Linear resolution:**

None

```
[ C4: Bat(Brown_Bat) ]  (Starting Center Clause)

         |

         ├─── [ C1: ¬Bat(x) ∨ HasWings(x) ] (Side Clause)

         ▼

[ C6: HasWings(Brown_Bat) ] (New Center Clause)
```

```
          |
          ├──── [ C3: ¬HasWings(z) ∨ ¬HasBarrel(z) ] (Side Clause)
          ▼
[ C8: ¬HasBarrel(Brown_Bat) ] (New Center Clause)
          |
          ├──── [ C2: ¬Bat(y) ∨ HasBarrel(y) ] (Side Clause)
          ▼
[ C9: ¬Bat(Brown_Bat) ] (New Center Clause)
          |
          ├──── [ C4: Bat(Brown_Bat) ] (Side Clause - reuse of original)
          ▼
   [ Empty Clause (□) ]
```

Steps for Linear Resolution explained in detail:

1. $\frac{Bat(Brown_Bat), \neg Bat(Brown_Bat) \vee HasWings(Brown_Bat)}{HasWings(Brown_Bat)}$
2. Maintaining the chain: Center Clause: $HasWings(Brown_Bat)$ -> previously derived clause

3. $\frac{HasWings(Brown_Bat), \neg HasWings(Brown_Bat) \vee \neg HasBarrel(Brown_Bat)}{\neg HasBarrel(Brown_Bat)}$
4. $\frac{\neg HasBarrel(Brown_Bat), \neg Bat(Brown_Bat) \vee HasBarrel(Brown_Bat)}{\neg Bat(Brown_Bat)}$
5. $\frac{\neg Bat(Brown_Bat),\ Bat(Brown_Bat)}{\square}$